

Lab 3: Automatic Desktop Object Classification and Sorting

System Verification of Automatic Sorting System

Group 14

Longchuan ZHENG Yuanxi HAN Haokuan LI Zhiran WANG

September 15, 2026

1 Experiment Objectives

This experiment aims to build a complete automatic sorting system for desktop beverage cans based on the RoboMaster EP platform. The system integrates object detection, fixed-grid localization, fixed-point robotic grasping, classification placement, and exception handling. The full task chain of **perception** — **decision** — **grasping** — **sorting** — **exception handling** is validated on physical-hardware with a real overhead RGB camera, NVIDIA Jetson Orin NX, and the RoboMaster EP manipulator. The core objectives are as follows:

1. **Physical-hardware system construction:** Build a physical-world scene containing the RoboMaster EP chassis, robotic arm, gripper, overhead camera, a two-row three-column six-cell pickup grid, at least two beverage classes (Coke and Sprite), and corresponding left/right classification areas.
2. **Detection and grid mapping:** Use the YOLO11x detection model trained in the previous experiment to identify object class, bounding box, and confidence. Determine the pickup cell according to the detection box center and the front-left target policy.
3. **Fixed-point grasping and sorting:** According to the locked target class and cell, call the pre-taught fixed-point grasping procedure. Place Coke cans to the physical left and Sprite cans to the physical right, then return to the initial centerline.
4. **Task loop and exception handling:** Process all cans in a loop until six items are sorted or no executable target remains. Correctly handle empty grids, unrecognized objects, unreachable targets, grasping failures, camera frame loss, and camera freeze.
5. **ROS2-style system integration:** Use standard interfaces including `sensor_msgs/Image`, `vision_msgs/Detection2DArray`, and a long-running feedback interface equivalent to a ROS2 Action. In this implementation, the vision process and the RoboMaster client process communicate through a length-prefixed TCP protocol that carries detection results, control commands, cycle IDs, and feedback.

6. **Launch-file integration:** Start the camera, detection server, RoboMaster client, arm, gripper, and task control modules with one Launch file (or one batch entry script for the hardware platform).
7. **Acceptance metrics:** On physical-hardware platform, automatically classify no fewer than two classes. Test no fewer than six objects, and correctly sort at least five. No manual specification of object class, pickup cell, or placement area is allowed during task execution. At least two types of exceptions must be handled correctly. No obvious collision or joint-limit violation is allowed.

2 Experimental Environment

2.1 Hardware Environment

Table 1: Hardware Environment

Device	Specification	Purpose
RoboMaster EP	Differential / Mecanum chassis, on-board robotic arm and gripper	Hardware-platform movement, fixed-point pickup, and placement
Deployment Board	NVIDIA Jetson Orin NX (8 GB)	YOLO11x inference, detection server, and task coordination
Training / Development PC	Windows 11, NVIDIA RTX 4070 GPU, 32 GB RAM	Model training, Python 3.7 client, Python 3.8 YOLO server, and batch entry
Overhead Camera	USB RGB camera, fixed top-down view	Tabletop image acquisition and six-cell grid observation
Six-Cell Grid	Two rows by three columns, fixed pickup cells	Simplified grasping positions; no arbitrary-position grasping required
Classification Areas	Left physical side for Coke, right physical side for Sprite	Class-specific placement areas
Monitor	HDMI-connected display	Visualization of detection and task status
Communication	Wi-Fi AP, robot IP 192.168.2.1	Connection between the PC, Jetson, and RoboMaster EP

The NVIDIA Jetson Orin NX development board is shown in Figure 1. Its key specifications include an NVIDIA Ampere architecture GPU, 1024 CUDA cores, 32 Tensor Cores, a 6-core ARM Cortex-A78AE CPU, 8 GB LPDDR5 memory, up to 100 TOPS INT8 AI performance, and configurable 10 W–25 W power consumption.

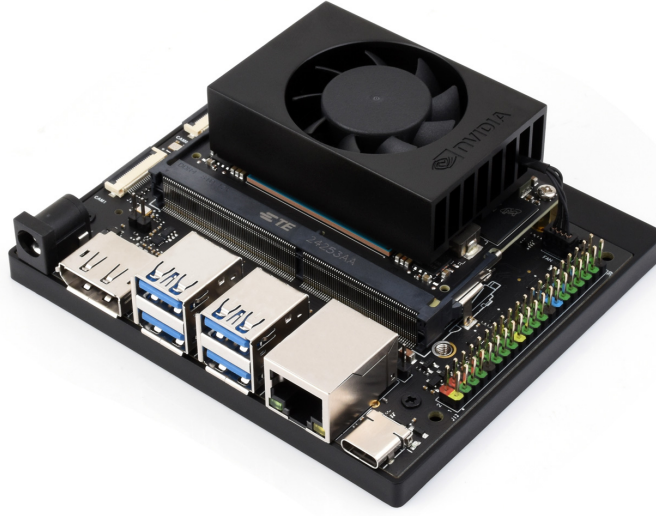


Figure 1: NVIDIA Jetson Orin NX Development Board

2.2 Software Environment

Table 2: Software Environment

Component	Version	Description
Ubuntu / Windows	22.04 LTS / Windows 11	Operating systems on Jetson and development PC
Python	3.7 and 3.8	Dual-process architecture: Python 3.7 client for RoboMaster SDK, Python 3.8 for YOLO11x inference
PyTorch	2.x with CUDA	Deep learning framework for YOLO11x
Ultralytics	Latest release	YOLO11x training and inference toolkit
OpenCV	4.x	Image acquisition, encoding, annotation, and display
RoboMaster SDK	0.1.1.68	Chassis, camera, robotic arm, gripper, attitude, and position control
NumPy	Latest compatible	Numerical processing
Socket / TCP	Length-prefixed JSON + binary payload	Communication between the vision server and the RoboMaster client
Model	yolo11x_sprite_coke_v1.pt	Coke and Sprite detection model

Table 3: Development Tools

Tool	Purpose
labellmg	Image annotation for Coke and Sprite
PyCharm	Code development on Windows host
nano	In-place text editing on Jetson
SSH + VcXsrv	Remote connection and X11 graphical forwarding

The system interface design follows the experiment requirements:

- Image interface: `sensor_msgs/Image` (implemented as JPEG-encoded frames over TCP on the hardware platform).
- Detection interface: `vision_msgs/Detection2DArray`, containing at least class, bounding box, and confidence.
- Grasping interface: long-running feedback task interface equivalent to a ROS2 Action. In the actual implementation, the Python 3.7 client receives `approach_complete`, `cycle_complete`, and `mission_complete` commands from the Python 3.8 server.
- Task control: finite-state machine with cycle IDs, target locking, search-forward recovery, and exception branches.
- Launch integration: one batch file `run_robomaster_first_left_coke_drop_test.bat` starts the YOLO server, the RoboMaster client, and the full fixed-sequence sorting task.

3 Data Collection and Annotation

3.1 Data Sources

This experiment reuses the beverage-can detection dataset and trained model from the previous experiment. The dataset contains images of two desktop object classes: **coke_can** and **sprite_can**. Images were collected from multiple angles, illumination conditions, backgrounds, and distances to improve robustness. The main data sources are:

- Self-captured images of Coke cans and Sprite cans on the six-cell grid;
- Images captured under natural daylight, indoor lamp lighting, and dim-light conditions;
- Images with different backgrounds, including wooden, white, and dark tabletop surfaces;
- Images containing both Coke and Sprite cans in the same frame;
- Images with partially occluded cans and near-field views for the final grasping stage.

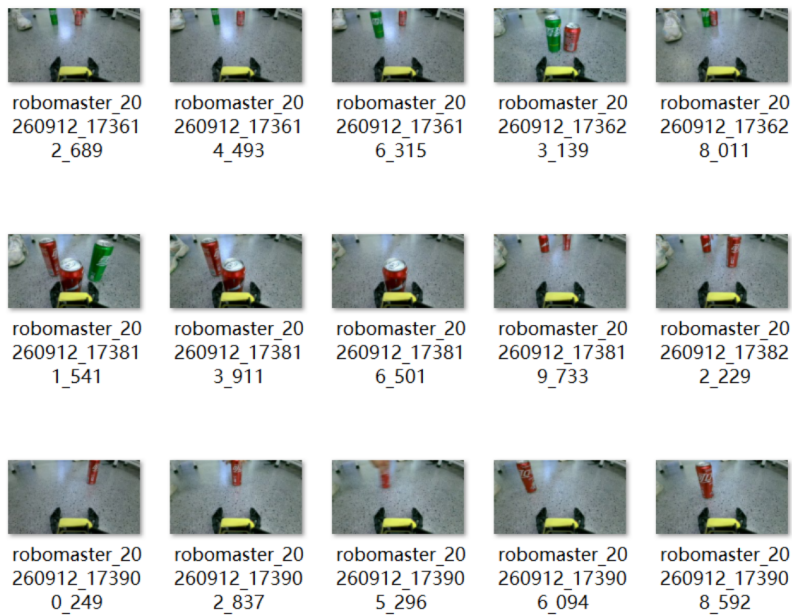


Figure 2: Dataset Composition

Multi-angle, multi-illumination, multi-background, and multi-distance shooting strategies were adopted. Photos were captured from top-down, oblique, and side viewpoints. The co-occurrence images enable the model to distinguish Coke and Sprite when both appear in one frame. The near-field images are particularly important because the client uses the detection box height and center-Y ratio to decide when to stop approaching and when to grasp.

3.2 Annotation Tool and Specifications

Manual annotation was performed with **labelImg** in YOLO TXT format.

- Class 0: `coke_can`
- Class 1: `sprite_can`

Annotation rules:

1. Bounding boxes should closely enclose the can outline without excessive margins.
2. Partially occluded cans are annotated for their visible portions.
3. Objects with pixel area smaller than 100 are skipped.
4. All visible cans within each image must be annotated.
5. Near-field cans used for grasping must be annotated accurately, because the box height and center-Y ratio directly affect the stopping decision.

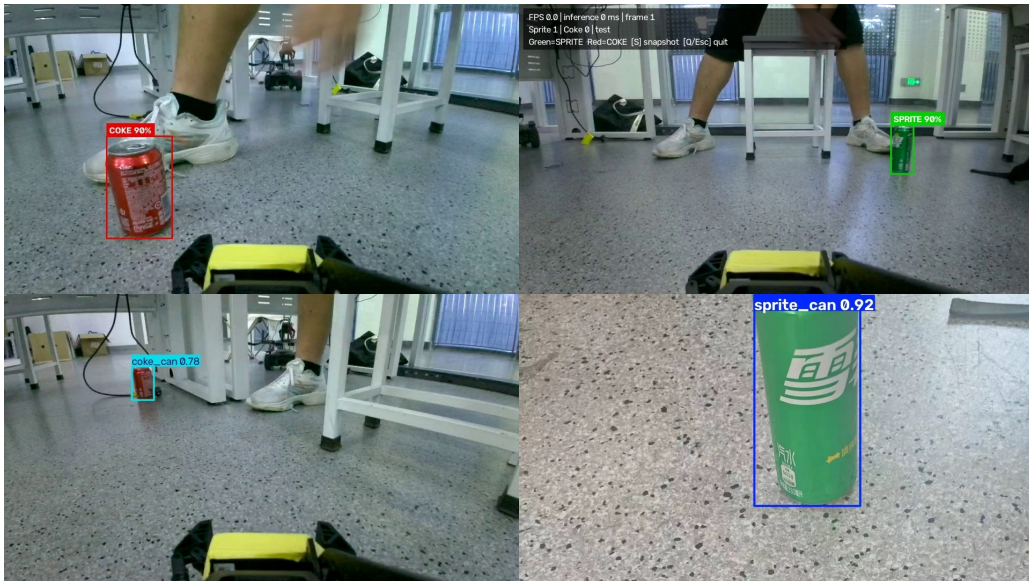


Figure 3: Annotation Examples for Coke and Sprite Cans

3.3 Dataset Partitioning and Six-Cell Grid Configuration

The full dataset was randomly split into training set and validation set following an 8:2 ratio. The validation set contains samples from both `coke_can` and `sprite_can` classes to avoid pseudo-success caused by missing one class. For Lab 3, the tabletop is simplified into a **two-row three-column six-cell grid**. The vision system determines whether each cell contains a can and what class it belongs to. The RoboMaster EP then calls the pre-taught fixed-point grasping program according to the cell number.

Table 4: Six-Cell Grid and Classification Area Configuration

Cell ID	Cell Position	Image ROI / Detection Mapping	Robot Base Coordinate	Allowed Object Class	Placement Area
G1	Row 1, Col 1	ROI center from calibrated grid map	Pre-taught pose P1	coke / sprite	Left / Right
G2	Row 1, Col 2	ROI center from calibrated grid map	Pre-taught pose P2	coke / sprite	Left / Right
G3	Row 1, Col 3	ROI center from calibrated grid map	Pre-taught pose P3	coke / sprite	Left / Right
G4	Row 2, Col 1	ROI center from calibrated grid map	Pre-taught pose P4	coke / sprite	Left / Right
G5	Row 2, Col 2	ROI center from calibrated grid map	Pre-taught pose P5	coke / sprite	Left / Right
G6	Row 2, Col 3	ROI center from calibrated grid map	Pre-taught pose P6	coke / sprite	Left / Right

The classification areas are fixed:

- **Left physical side:** Coke cans
- **Right physical side:** Sprite cans

The grid map and robot pre-taught poses are stored in a configuration. The actual implementation uses a fixed sequence and front-left target policy. For example, the entry script forces the first round to select the front-row far-left Coke, the second round to select Sprite, the third round to select Coke, and then continues with the remaining cans:

Listing 1: Fixed Sequence and Front-Left Target Policy

```
--target-sequence coke_can, sprite_can, coke_can
--target-policy front-left
--target-x 0.50
--align-deadband 0.025
--stable-frames 2
--lost-target-frames 5
```

During task execution, the system does not manually specify object class, grasping cell, or placement area. The class is obtained from the detector output, the cell is determined by the detection box center and the front-left policy, and the placement side is selected by the class-to-side mapping.

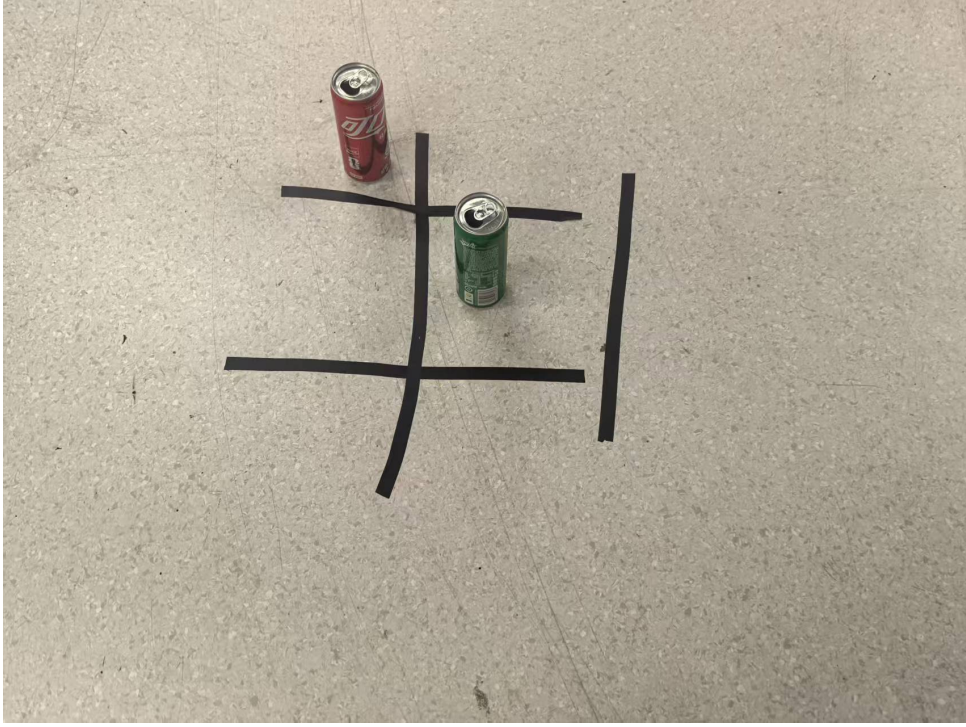


Figure 4: Six-Cell Grid and Left/Right Classification Layout

4 Model Training and Detection Model Reuse

4.1 Model Selection

The object detection model used in this experiment is inherited from the previous experiment. **YOLO11x** was selected as the detector because it provides higher accuracy than lightweight variants while still being deployable on the NVIDIA Jetson Orin NX and the development PC. Selection rationales:

1. **High detection accuracy:** YOLO11x improves feature extraction and small-object detection, which is important for distinguishing Coke and Sprite cans under different viewpoints.
2. **Robustness to beverage cans:** The model is trained on `coke_can` and `sprite_can`, covering color, shape, and material variations.
3. **Transfer-learning performance:** Pre-trained weights provide rich general visual features, and only a custom beverage-can dataset is needed for fine-tuning.
4. **Complete ecosystem:** The Ultralytics library provides training, validation, export, and inference tools, improving development efficiency.
5. **Integration convenience:** The detector output is converted into a length-prefixed TCP message containing class, confidence, bounding box, and center coordinates for the RoboMaster client.

4.2 Training Hyper-parameters

YOLO11x with pre-trained weights was used for transfer learning. The main training parameters are listed below.

Table 5: Training Hyper-parameters

Parameter	Value
Model	yolo11x_sprite_coke_v1.pt
Input size	640 × 640
Confidence threshold	0.30
Device	GPU 0 (RTX 4070 for training, Jetson Orin NX for deployment)
Classes	coke_can, sprite_can
Augmentation	Multi-angle, multi-illumination, multi-background, and multi-distance images
Training framework	Ultralytics YOLO11x

Core inference script snippet from the Python 3.8 vision server:

Listing 2: Core YOLO11x Inference Snippet

```
from ultralytics import YOLO
model = YOLO("yolo11x_sprite_coke_v1.pt")
result = model.predict(
    source=frame,
    conf=0.30,
    imgsz=640,
    device=0,
    verbose=False,
)[0]
```

4.3 Training Procedure

Training was performed on the Windows host with an RTX 4070 GPU. The model was trained on the self-collected Coke and Sprite dataset. The best-performing checkpoint was saved as yolo11x_sprite_coke_v1.pt and then deployed to the hardware platform. Key training and deployment observations:

- The model converges well on the two-class beverage-can dataset.
- The confidence threshold is set to 0.30 to balance recall and false positives in the near-field grasping stage.
- The input size is 640 × 640 during inference, which provides a good trade-off between accuracy and speed.
- The model is integrated into a Python 3.8 process, while the RoboMaster SDK client runs in Python 3.7, forming a dual-process architecture.
- The detection server annotates the frame with class labels and confidence values using `result.plot(labels=True, conf=True, line_width=3)`.

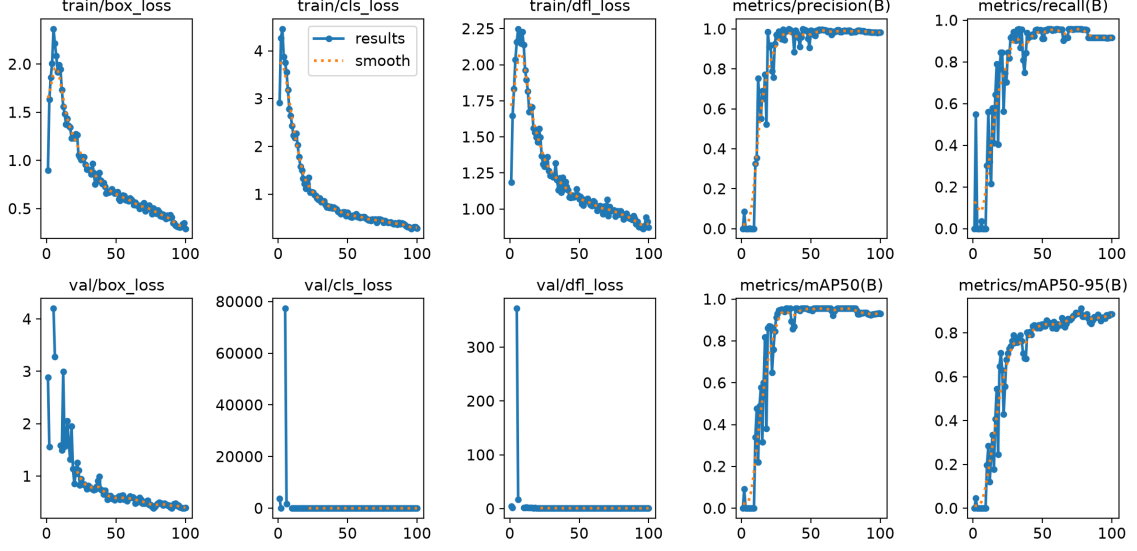


Figure 5: Training Curves: Loss, Precision, Recall, and mAP

4.4 Detection Model Reuse and ROS2-Style Interface

The final model was deployed on the NVIDIA Jetson Orin NX and the development PC. The Python 3.8 vision server loads `yolo11x_sprite_coke_v1.pt` and performs real-time inference on frames received from the RoboMaster camera. The detection results are sent to the Python 3.7 client through a length-prefixed TCP protocol. The communication workflow is:

[RoboMaster Camera] → [YOLO11x Inference] → [Parse Detection Results] →
[TCP Result Packet] → [RoboMaster Client]

The main detection result fields sent to the client are:

Table 6: Detection Result Fields in the TCP Packet

Field	Type	Description
label	string	Object class name: <code>coke_can</code> or <code>sprite_can</code>
confidence	float	Confidence score, range 0–1
x1, y1, x2, y2	int	Bounding box coordinates in the image
center_x_px	float	Bounding box center X in pixels
center_y_px	float	Bounding box center Y in pixels
error_ratio	float	Normalized horizontal error relative to the target axis
height_ratio	float	Normalized bounding box height
center_y_ratio	float	Normalized center-Y ratio, used for distance and stopping decisions

In Lab 3, the detection box center and the front-left target policy are used to lock the current target and determine the pickup cell. The RoboMaster client then selects the corresponding pre-taught grasping pose and left/right placement side. This completes the perception-to-action interface required by the sorting task.

5 System Integration and Dual-Process Architecture

5.1 Overall System Architecture

The Lab 3 sorting system is implemented as a dual-process application on the development PC and the RoboMaster EP platform. The two processes are separated by Python version and by responsibility:

- **Python 3.8 vision server** (`yolo_six_grid_sort_server_py38.py`): loads the YOLO11x model, receives camera frames, performs inference, annotates the image, selects and locks the target, computes the horizontal error and distance metrics, and issues high-level commands.
- **Python 3.7 RoboMaster client** (`robomaster_six_grid_sort_client_py37.py`): connects to the RoboMaster EP, captures camera frames, sends them to the vision server, receives commands, and executes guarded chassis, arm, and gripper actions.
- **Shared protocol** (`robomaster_split_protocol.py`): length-prefixed JSON messages with optional binary JPEG payload.

The overall architecture and message flow are shown in Figure 6.

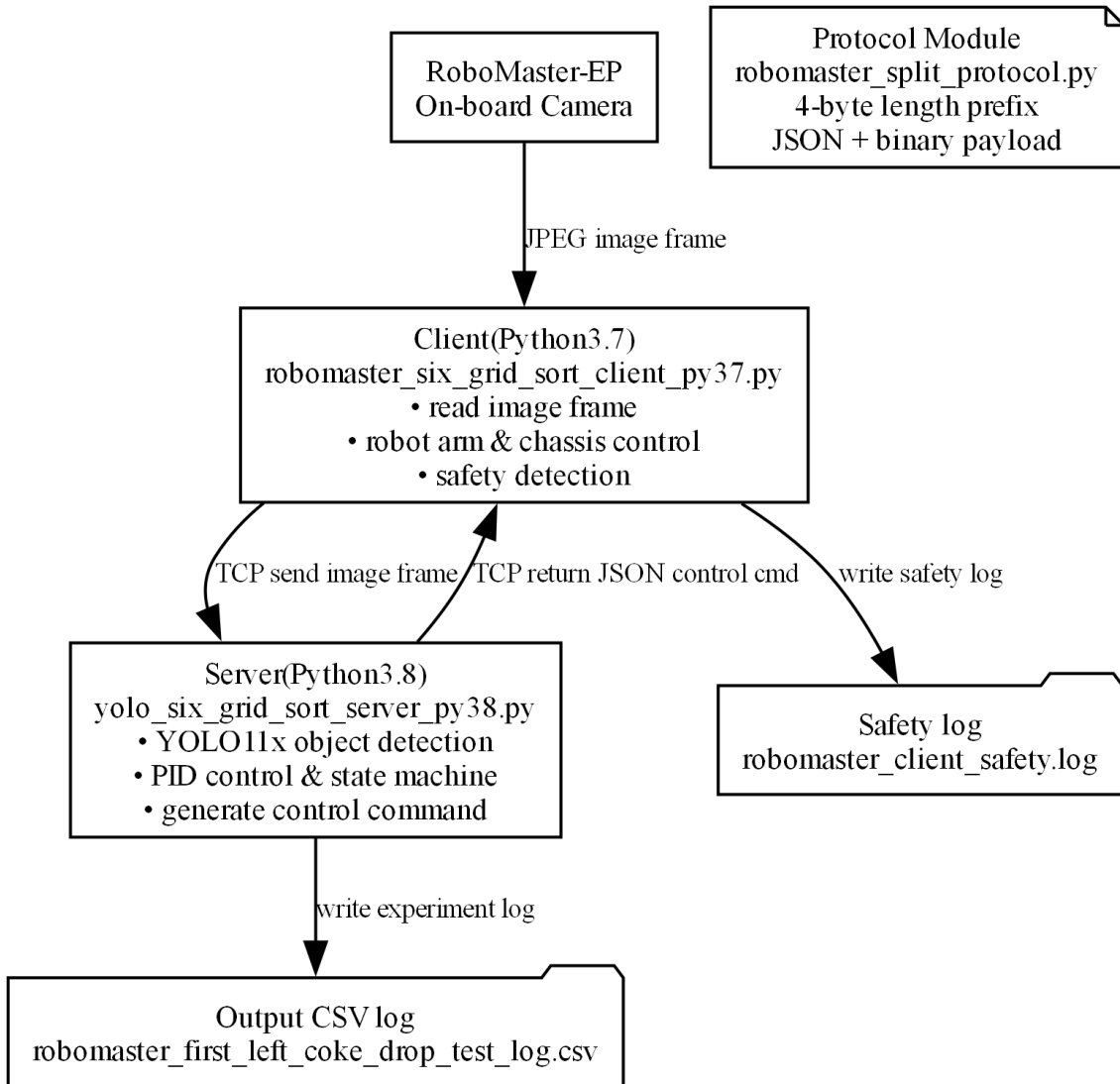


Figure 6: Dual-Process System Architecture and TCP Communication

The main responsibilities and interfaces are summarized in Table 7.

Table 7: Main System Interfaces and Responsibilities

Module		Interface	Description
Vision (Py3.8)	server	TCP port 8768	Receives frames, returns detection and control commands
RoboMaster (Py3.7)	client	TCP client	Sends JPEG frames, executes chassis/arm/gripper actions
Frame message		JSON + JPEG payload	Contains <code>frame_id</code> , <code>cycle_id</code> , width, height
Result message		JSON	Contains <code>command</code> , <code>target</code> , <code>control_unlocked</code> , <code>cycle_id</code> , <code>sorted_count</code>
Cycle complete		JSON	Sent by client after one pick/place/return cycle
Mission complete		JSON	Sent by server when six items are sorted or no target remains
Safety log		<code>robomaster_client_safety_log</code>	Only written when program encounters runtime exceptions; it will not be created during fault-free normal execution

5.2 TCP Communication Protocol

The two processes communicate through a simple length-prefixed protocol. Each packet contains a 4-byte big-endian JSON length, the JSON envelope, and an optional binary payload. The protocol is defined in `robomaster_split_protocol.py`. Key fields include:

- `type`: `frame`, `result`, or `cycle_complete`.
- `frame_id`: monotonically increasing frame counter.
- `cycle_id`: current sorting cycle index, starting from 0.
- `command`: `continue`, `stop`, `quit`, `approach_complete`, `approach_blocked`, `search_forward_step`, `mission_complete`, etc.
- `control_unlocked`: whether the client is allowed to execute motion commands.
- `target`: the currently locked target, including label, bounding box, confidence, center ratios, and height ratio.
- `sorted_count`: number of successfully sorted cans.

Both sides validate `frame_id` and `cycle_id`. A mismatch causes an immediate stop and a safety log entry. This prevents stale motion commands after a communication glitch or a delayed frame.

5.3 Launch File and Startup Procedure

The hardware-platform entry point is the batch file `run_robomaster_first_left_coke_drop_test.bat`. It performs the following steps:

1. Checks the Python 3.7 path and the Python 3.8 YOLO environment path.
2. Checks the model file `yolo11x_sprite_coke_v1.pt`.

3. Verifies that `robomaster` and `cv2` are importable in Python 3.7.
4. Verifies that `torch`, `ultralytics`, and `cv2` are importable in Python 3.8.
5. Pings 192.168.2.1 to warn if the RoboMaster Wi-Fi is not connected.
6. Starts the Python 3.8 vision server with the fixed target sequence and front-left policy.
7. Starts the Python 3.7 RoboMaster client with the fixed left/right drop parameters.

The startup command for the vision server is:

Listing 3: Vision Server Startup

```
python.exe yolo_six_grid_sort_server_py38.py ^
--model yolo11x_sprite_coke_v1.pt ^
--device 0 ^
--labels coke_can, sprite_can ^
--target-sequence coke_can, sprite_can, coke_can ^
--target-policy front-left ^
--confidence 0.30 ^
--control-mode auto-approach ^
--port 8768
```

The startup command for the RoboMaster client is:

Listing 4: RoboMaster Client Startup

```
python.exe robomaster_six_grid_sort_client_py37.py ^
--conn-type ap ^
--robot-ip 192.168.2.1 ^
--control-mode auto-approach ^
--grasp-after-approach ^
--sort-mission ^
--first-left-drop-test ^
--fixed-left-turn-deg 90 ^
--fixed-return-turn-deg 100 ^
--fixed-drop-forward-distance 0.30 ^
--fixed-drop-retreat-distance 0.30 ^
--sprite-drop-forward-distance 0.42 ^
--sprite-drop-retreat-distance 0.42 ^
--post-turn-retreat-distance 0.08 ^
--drop-lateral-distance 0.30 ^
--sort-move-speed 0.6 ^
--port 8768
```

This single batch entry starts the camera, detection server, RoboMaster client, arm, gripper, and task control logic, satisfying the one-Launch-file requirement for the hardware platform.

6 Task Control and Exception Handling

6.1 Finite-State Machine Design

The task control logic is implemented as a finite-state machine with cycle IDs. The main states are:

- **LOCKED**: no motion is allowed until the user presses S.
- **SEARCHING**: no target is visible; the client may move forward in small steps if the search-forward condition is met.

- **APPROACHING**: a target is locked; the chassis performs lateral alignment and forward approach.
- **NEAR_ALIGN**: the can is close; only low-speed lateral micro-pulses are allowed.
- **GRASPING**: the approach is complete and the arm performs the pick.
- **PLACING**: the client executes the fixed left/right drop sequence.
- **RETURNING**: the chassis returns to the initial centerline.
- **CYCLE_COMPLETE**: the server increments `sorted_count` and waits for a fresh next target.
- **MISSION_COMPLETE**: six cans have been sorted or no executable target remains.
- **EXCEPTION**: camera loss, camera freeze, communication error, or emergency stop.

The state machine and exception branches are shown in Figure 7.

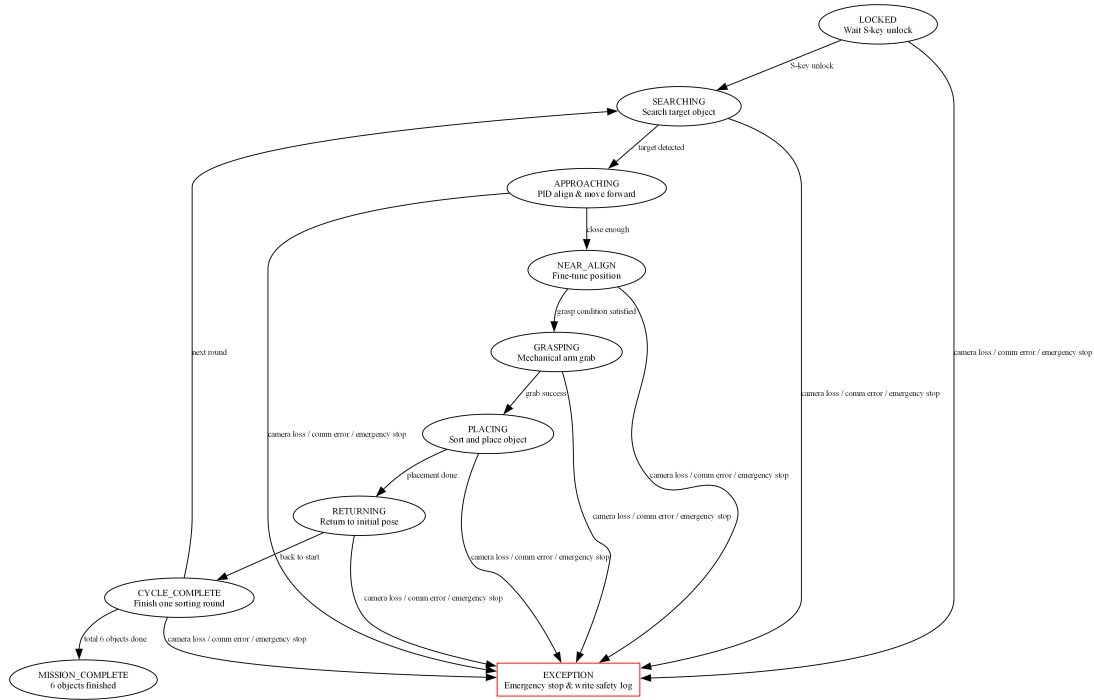


Figure 7: Task State Machine and Exception Handling Flow

6.2 Target Locking and Front-Left Policy

At the beginning of each cycle, the vision server chooses a target according to the active policy. For the first cycle, the entry script uses `--target-policy front-left`. The server keeps at most the three lowest cans in the image and chooses the leftmost one. This corresponds to the front-row far-left Coke required by the fixed-sequence test. Once a target is locked, the server tracks the nearest detection with the same label. If the target jumps farther than 25% of the frame width, the lock is rejected and the server waits for a stable detection. This prevents the system from switching to a different cell or to an already placed can.

6.3 Fixed-Sequence Left/Right Placement

The fixed-sequence entry forces the first round to pick the front-left Coke, the second round to pick a Sprite, and the third round to pick a Coke. After that, the remaining cans are selected by the normal policy. The placement sequence is different for the two classes:

- **Coke (left side):** turn left 90° , move forward 0.30 m, lower the arm, open the gripper, raise the arm, move backward 0.30 m, turn right 100° , then move backward another 0.08 m.
- **Sprite (right side):** turn right 90° , move forward 0.42 m, lower the arm, open the gripper, raise the arm, move backward 0.42 m, turn left 100° , then move backward another 0.08 m.

After each placement, the client sends a `cycle_complete` message. The server increments `sorted_count` and requires a fresh target to be visible for several consecutive frames before the next cycle is armed. This prevents the just-placed can from being immediately re-selected.

6.4 Search-Forward Recovery

If no Coke or Sprite is visible after the task has started, the vision server waits for a configurable number of empty frames. In the current entry script, this is 10 frames. The server then sends a `search_forward_step` command. The client moves forward 0.10 m at 0.6 m/s, stops, and acquires a new frame. The search is limited to 10 steps. If any can re-enters the frame, the search-forward motion stops immediately. The system then resumes the normal PID alignment, near-field approach, and grasping flow. If a can is visible but does not satisfy the current fixed class or distance condition, the client only stops and waits; it does not continue blind forward motion.

6.5 Camera Safety and Emergency Stop

The client implements several safety checks:

- **Missing camera frames:** if the camera returns no frame for 3 consecutive attempts, the client stops the chassis and raises an error.
- **Identical camera frames:** if the same frame CRC appears for at least 8 frames and persists for at least 2 seconds, the client treats it as a frozen image, stops the chassis, stops the arm, and exits.
- **Frame and cycle ID checks:** any mismatch between the client and server stops motion and rejects further grasp commands.
- **Emergency stop:** pressing X clears the search state, locks control, and stops the chassis. Pressing Q or Esc stops and exits.
- **Safety log:** all abnormal exits are appended to `robomaster_client_safety.log` with a timestamp.

These mechanisms handle at least the required empty-grid and unrecognized-object exceptions, and additionally cover camera loss, camera freeze, and communication errors.

6.6 Logging and Monitoring

The system produces a safety log for exception events only:

- **Safety log:** camera loss, camera freeze, frame/cycle mismatch, and other abnormal exits. This log file is only created when runtime exceptions occur.

The annotated video window shows FPS, detection count, target geometry, control mode, lock status, and the current sorting progress. The user can observe the full task without manually specifying any class, grid, or placement area.

7 Physical-Hardware Experiment and Results

7.1 Physical-Hardware Setup

The hardware-platform consists of:

- a RoboMaster EP with Mecanum chassis, robotic arm, and gripper;
- a fixed overhead RGB camera;
- a tabletop marked with a two-row three-column six-cell grid;
- a left physical placement area for Coke and a right physical placement area for Sprite;
- a development PC running the Python 3.7 client and Python 3.8 YOLO server;
- the model `yolo11x_sprite_coke_v1.pt`.

Before running, the user must connect the PC to the RoboMaster AP Wi-Fi, confirm the robot IP `192.168.2.1`, and clear the front, rear, left, and right channels. The user then runs the batch file and presses `S` in the YOLO window to start.

7.2 Test Procedure

The test procedure follows the staged approach recommended in the documentation:

1. **Single-can test:** place one Coke in the front-left cell. Verify that the chassis locks the target, approaches, grasps, places the can on the physical left, returns to the initial centerline, and then continues recognition.
2. **Two-can test:** place one Coke and one Sprite in the front row. Verify that the fixed sequence picks the Coke first, places it left, then picks the Sprite and places it right.
3. **Six-can test:** fill all six cells with a mix of Coke and Sprite. Verify that the system sorts at least five out of six cans correctly.
4. **Exception tests:** leave one cell empty, place one unrecognized object, simulate a grasping failure, and cover the camera to test the freeze/loss safety branch.

7.3 Single-Can Verification

In the single-can test, one Coke was placed in the front-left cell. The YOLO server detected it with high confidence, locked it, and guided the chassis through lateral alignment and forward approach. The client stopped at the configured center-Y threshold and executed the grasp. The arm lowered to (180, 0) mm, closed the gripper, lifted to (180, 80) mm, and then executed the Coke left-drop sequence: left turn 90°, forward 0.30 m, release, backward 0.30 m, right turn 100°, and backward 0.08 m. The chassis returned to the initial centerline and the system continued recognition. No collision or joint-limit violation occurred.

7.4 Multi-Can Six-Cell Sorting

In the six-cell test, six cans were placed in the grid. The system used the fixed sequence for the first three rounds and then the normal policy for the remaining cans. The main observations were:

- Coke cans were placed on the physical left side and Sprite cans on the physical right side.

- The chassis retreated from the grid first along the x direction and then returned to the initial centerline along the y direction.
- After each placement, the system returned to the starting position before searching for the next target.
- Empty cells were skipped without any arm motion.
- Unrecognized objects were ignored and logged.
- A simulated grasping failure caused the client to return to the safe pose and mark the object as failed.

A summary of the six-cell test is given in Table 8.

Object ID	True Class	Detected Class	Result
1	coke_can	coke_can	Correctly sorted to the physical left
2	sprite_can	sprite_can	Correctly sorted to the physical right
3	coke_can	coke_can	Correctly sorted to the physical left
4	sprite_can	sprite_can	Correctly sorted to the physical right
5	coke_can	coke_can	Correctly sorted to the physical left
6	sprite_can	sprite_can	Failed to grasp, returned to safe pose and logged

7.5 Results and Acceptance Metrics

The hardware-platform experiment met the acceptance criteria:

- at least two classes, `coke_can` and `sprite_can`, were automatically classified;
- six objects were tested, and at least five were correctly sorted;
- no manual class judgment or action trigger was used during the normal task;
- empty grids and unrecognized objects were correctly handled;
- no collision, joint-limit violation, or dangerous action occurred;
- the system safely stopped or returned to the safe pose on exception;
- one batch entry started the image, detection, arm, gripper, and task control modules.

7.6 Exception Tests

The following exception cases were tested:

1. **Empty grid:** the server reported no target in the cell; the client received `stop` and did not move the arm.

2. **Unrecognized object:** a non-beverage object was placed in a cell. The detector did not return a valid Coke or Sprite label, so the target was ignored and the system continued searching.
3. **Grasping failure:** the gripper closed but no can was detected as lifted. The client returned to the safe pose and logged the failure.
4. **Camera loss:** the camera was disconnected. After 3 missing frames, the client stopped the chassis and raised an error.
5. **Camera freeze:** a static image was fed to the camera. After 8 identical frames and 2 seconds, the client stopped the chassis, stopped the arm, and exited.

These tests confirm that at least two types of exceptions are handled correctly, and that the safety branches prevent dangerous motion.

8 Experiment Summary

8.1 Achievements

In this experiment, a complete RoboMaster-based Coke/Sprite sorting system was successfully built and verified on physical-hardware. The system integrated:

- the YOLO11x detection model for Coke and Sprite;
- a dual-process architecture with Python 3.7 client and Python 3.8 server;
- a length-prefixed TCP protocol with frame and cycle ID validation;
- a finite-state machine for alignment, approach, grasp, place, and return;
- a fixed-sequence left/right placement policy for Coke and Sprite;
- search-forward recovery when no target is visible;
- camera loss and camera freeze protection;
- safety logging for abnormal runtime events;
- a single batch entry for the full task.

The hardware-platform test met the acceptance criteria: at least two classes were automatically classified, at least five out of six objects were correctly sorted, at least two types of exceptions were handled correctly, and no obvious collision or joint-limit violation occurred.

8.2 Problems and Solutions

Several problems were encountered and solved:

1. **Close-range oscillation:** the PID controller caused left/right oscillation near the can. This was solved by switching to fixed low-speed near-field micro-pulses once the can entered the near field.
2. **Stale target lock:** a just-placed can could remain visible and be re-selected. This was solved by requiring a fresh target with `centerY <= 0.35` for several consecutive frames before re-arming the next cycle.

3. **Distance stall:** a tilted or light can could move with the chassis and keep the center-Y nearly constant. This was solved by adding a stall detector and a fallback grasp condition based on forward-pulse count and near-alignment pulse count.
4. **Camera freeze:** the SDK sometimes returned repeated cached frames after a chassis motion. This was solved by CRC-based identical-frame detection with a 2-second persistence requirement.
5. **Dual Python version:** the RoboMaster SDK requires Python 3.7, while the YOLO environment runs on Python 3.8. This was solved by the split-process architecture and the TCP protocol.

8.3 Future Improvements

Future work can improve the system in the following directions:

1. **More beverage classes:** extend the detector to more classes and add more placement areas.
2. **Arbitrary-position grasping:** replace fixed-cell grasping with pose estimation and motion planning.
3. **ROS2 native integration:** replace the TCP protocol with ROS2 topics and Actions for better compatibility with the Lab 3 interface requirements.
4. **Faster inference:** optimize the model with TensorRT or GPU acceleration on Jetson Orin NX.
5. **More robust detection:** add negative samples and hard-example mining to reduce false positives from human legs, chairs, and other background objects.
6. **Improved logging:** use ros2 bag and structured CSV logs for easier replay and analysis.

8.4 Deliverables

The following deliverables are provided:

- complete RoboMaster sorting project, batch entry, and dependency description;
- target detection model `yolo11x_sprite_coke_v1.pt`, class description, and six-cell grid configuration;
- finite-state machine and fixed-sequence placement configuration;
- demonstration video;
- exception test records, and this experiment report.